

STM32 学习笔记

学习方法论：

看视频

视频相对而言是最轻松、有趣的学习方式

碰到不懂的可以先继续听，因为后面的东西前面用到了就讲了很正常

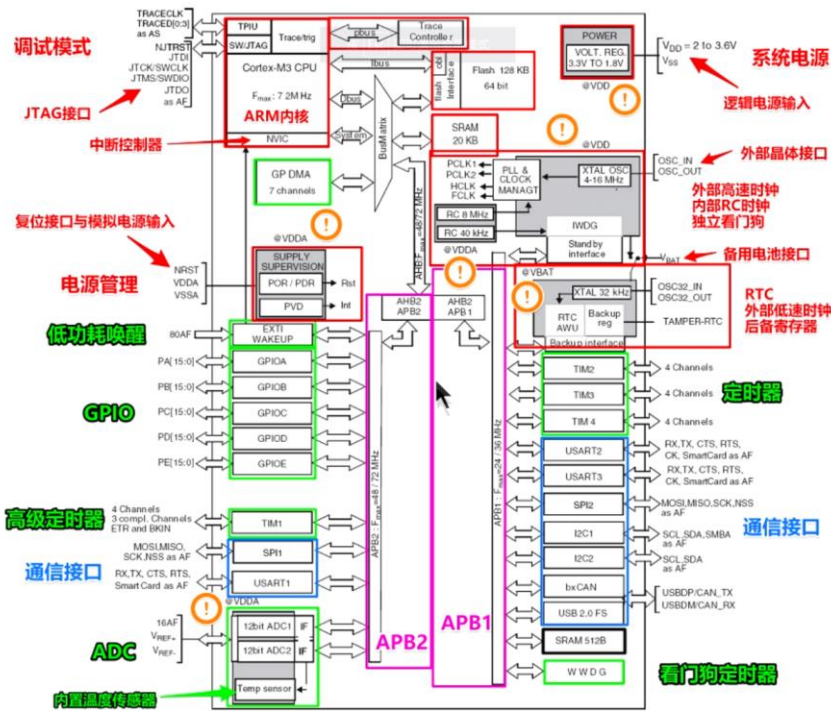
多听几遍，然后百度一下写笔记【一般视频都有配套资料，跟着一个资料系统走一遍】

看手册

数据手册（其他长篇文档也是）一定先通篇预览，再按需学习

数据手册主要用于芯片选型和设计原理图时参考，参考手册主要用于在编程的时候查阅

内部功能图



母线 busmatrix

[通过点亮 LED 灯全面理解寄存器编程](#)

编译出错解决办法

对于 error，可以双击则会自动跳到出错位置，在该位置上下几行寻找问题

实在找不到可以右键复制到剪贴板再百度求助

对于 warning，如果不影响运行有一些可以忽略（比如定义的变量未使用），而若影响运行则需要同 error 一样解决

基本概念

keil 的理解

Keil 是公司名，uVision 是集成开发环境（IDE），uVision5 是其中最新的一个版本
搭建环境，就是需要编程用的语言和用什么进行编程，用什么进行调试（调试就是找有没有 bug，所以调试器的英文叫做 debugger）的这几个条件的总和

在我们这儿，就是用 keil5 提供的软件以汇编语言进行编程，用开发工具进行调试

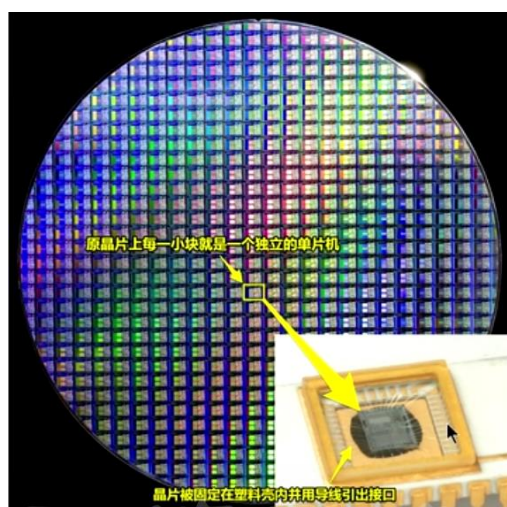
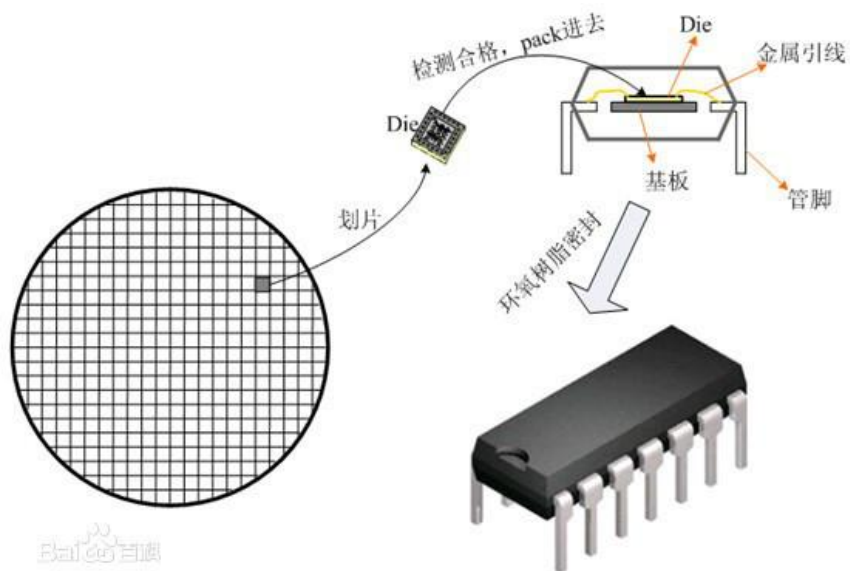
开发工具包括 MDK——支持 ARM 内核；c51——支持 8051 内核

软件本身的基础性作用就是把汇编语言编译生成单片机可执行的二进制代码，这样就可以烧写到单片机里面

[编程时搭建环境、搭建框架的含义详解](#)

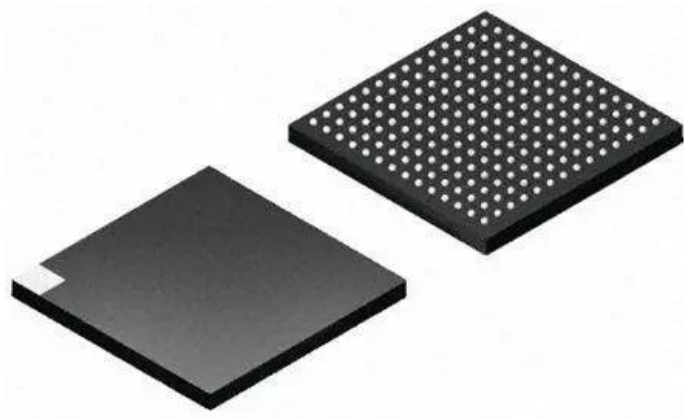
封装

F103 指的是设计的晶圆型号，晶圆经过激光蚀刻（形成上亿的晶体管）每一小块滑下来就是一个 MCU【感觉类似 AD 中的拼版】，晶圆取下来后就需要封装，下图就是双列直插式封装，连接金属引线的过程就是 bonding[绑定]



BGA, LQFP 是按照外形来划分的封装类型

这个是 BGA (底部有点)

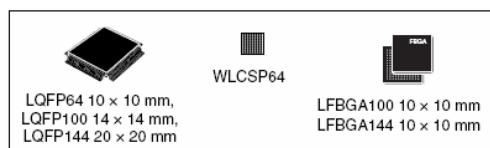


这个是 LQFP (四周有脚)，应用更为广泛



一般来说引脚少，功能少，内存也小，其他各方面均配套降低

图：数据手册中给出的封装使用说明



STM32 及 ARM

STM32 使用 ARM 内核 CPU

ARM 属于一个微控制器，STM32 自带了各种常用通信接口，比如 USART、I2C、SPI 等，

1、串口—USART，用于跟串口接口的设备通信，比如：USB 转串口模块、ESP8266

WIFI、GPS 模块，GSM 模块，串口屏、指纹识别模块

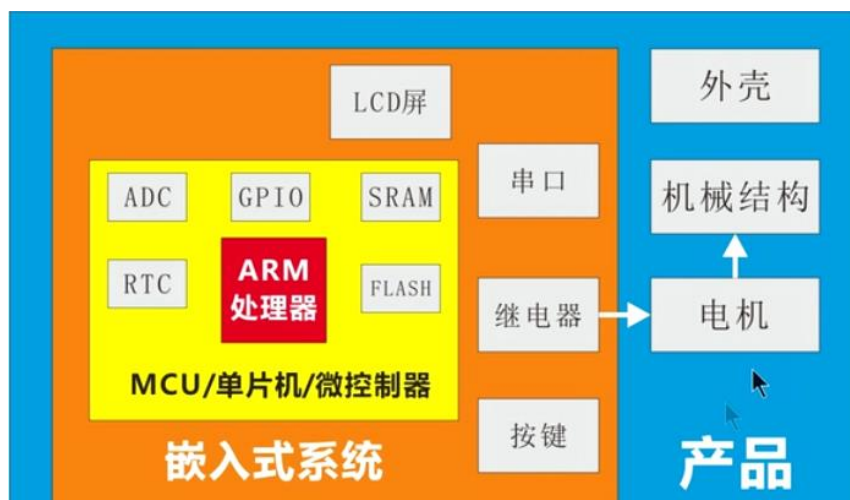
2、内部集成电路—I2C，用于跟 I2C 接口的设备通信，比如：EEPROM、电容屏、陀螺

仪 MPU6050、0.96 寸 OLED 模块

3、串行通信接口—SPI，用于跟 SPI 接口的设备通信，比如：串行 FLASH、以太网 W5500、音频模块 VS1053

4、SDIO、FSMC 的超级、I2S、ADC、GPIO

CPU,MCU,嵌入式系统联系与区别



应用层面

低成本

高性能

板卡层面

开发板

Arduino

树莓派

芯片层面

STC单片机 AVR单片机 STM32单片机

内核层面

8051内核

AVR内核

ARM内核

嵌入式系统

电脑是一个全能系统，但电脑成本太高。于是我们针对需要，只取电脑的一部分功能，对某样具有特定应用的电子产品进行自动控制、智能控制，这个用于控制的微电脑系统，就叫嵌入式系统。

注意被控制的产品本身不叫嵌入式系统，而只是在产品中完成自动控制、智能控制的微电脑系统的部分（包括硬件和软件）。

嵌入式系统最大的特点是根据产品的需要设计功能，没有多余的功能，最大限度降低成本。

单片机是嵌入式系统中最常用的核心部件。STM32本质上也是单片机。

控制电梯的是一种叫PLC（可编程控制器）的设备，但它对于电梯来说就是嵌入式系统。其实PLC的内部主要部件就是单片机，单片机对于PLC来说是嵌入式系统，PLC对于电梯而言是嵌入式系统。

如果用发条等机械装置做自动控制，这并不叫嵌入式系统，因为通常来说嵌入式系统是只电子电气控制。

例如：这台小水瓶是用热敏弹簧片来控制自动断电的，没有单片机，那就不算嵌入式系统。

早期很多电气是用数模电路实现自动控制，但这个不算嵌入式系统。嵌入式系统必须是由微电脑控制，也就是单片机。

嵌入式系统	硬件	单片机的电路板卡
	软件	嵌入式操作系统

MCU 选型

一个原则：花最少的钱，做最多的事

在确定项目需求的情况下，一般按照下面的顺序来选择合适的 MCU

- 1、选择哪种内核的芯片，内核越高意味着功耗也越高
- 2、选择多少引脚的芯片，引脚多少决定了资源的多少，也影响价格
- 3、选择多少 RAM 和 FLASH 的芯片，FLASH 越大，价格越贵
- 4、还要考虑所选型号采购是否容易，供货是否稳定

存储器概念辨析

存储器：

分类



内存一般指的 RAM，SRAM 无需刷新，所以速度比 DRAM 快，SRAM 一般只用于 CPU 内部的高速缓存(Cache)，而外部扩展的内存一般使用 DRAM【但是霸道扩展用的 SRAM】。

闪存指的是 FLASH，容量比 ROM 大

其中 NOR FLASH 一般应用在代码存储的场合

即我们平常所说的从 FLASH 读取到内存，可以简单粗暴的理解为从 NOR FLASH 读取到 SRAM
SD 卡、U 盘以及固态硬盘是 NAND FLASH

机械硬盘：

非易失性存储器种类非常多，半导体类的有 ROM 和 FLASH，而其它的则包括光盘、软盘及机械硬盘。

一般说的硬盘都是指的机械硬盘，硬盘应当是计算机的“外存”，储存空间(Storage)指的就是硬盘的容量

[各类型存储器区别与联系](#)

DAP 及串口 ISP

串口 isp 是成本低的下载方式，现在在 F4,7 系列已经很少使用，DAP 完全可以取代它（下载+调试+仿真）

debugger 直译为调试器，但是一般却叫做仿真器

高速版 HS——5M,全速版 FS——1M

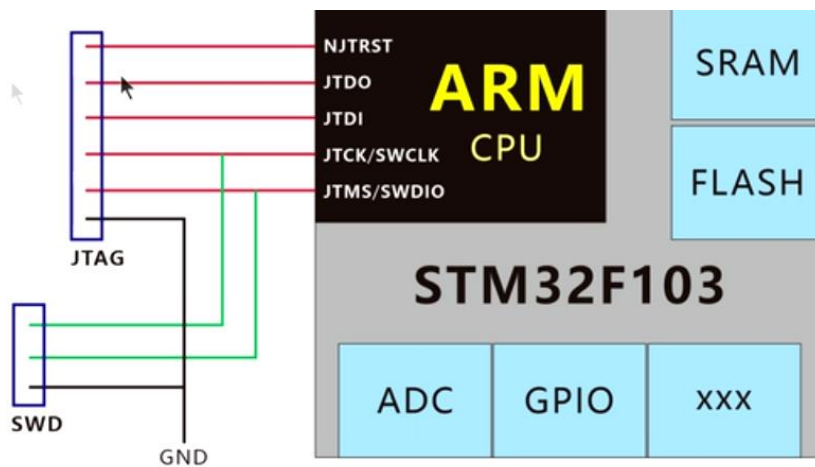
串口，JTAG，SW 均是下载模式的国际标准

高速版支持 JTAG 和 SW，全速版仅支持 SW【SW 和 SWD 是一样的】

内置 ARM 的 SWJ 接口，包含了 SW 和 JTAG 接口

内嵌ARM的SWJ-DP接口，这是一个结合了JTAG和串行单线调试的接口，可以实现串行单线调试接口或JTAG接口的连接。JTAG的TMS和TCK信号分别与SWDIO和SWCLK共用引脚，TMS脚上的一个特殊的信号序列用于在JTAG-DP和SW-DP间切换。

注意：ARM【内核】是采用了一种叫做 SWJ 的接口，包含了 SW 和 JTAG 两种接口；DAP【仿真器】是分成两种版本，支持不同接口



可见 JTAG 包含 SWD，SWCLK 是时钟线，SWDIO 是数据线，NJTRST 是复位线

串口【COM 口】功能只有下载程序

JTAG，SW 可在线调试和硬件仿真

[关于 JTAG，SW 的区别](#)

串口中 BOOT 就是引导的意思, Boot 模式设实际指的就是选择启动的起始地址区域, 在 STM32 中存在以下三种模式可供选择, 分别为片内 Flash、系统内存、片内 SRAM

2.3.8 自举模式

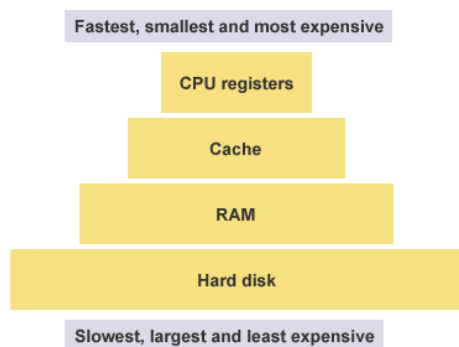
在启动时, 通过自举引脚可以选择三种自举模式中的一种:

- 从程序闪存存储器自举
- 从系统存储器自举
- 从内部SRAM自举

自举加载程序(Bootloader)存放于系统存储器中, 可以通过USART1对闪存重新编程。

寄存器

可以理解为 CPU 的零级缓存, 它内置于各个 IP 外设中, 是一种用于配置外设功能的存储器, 就是一种内存, 并且有相对应的地址。



存储器映射

存储器本身没有地址, 给存储器分配地址的过程叫存储器映射

寄存器映射

寄存器本身就有地址

给已经分配好地址的有特定功能的内存单元取别名的过程就叫寄存器映射。

如 `sfr p0=0x80`

野火寄存器点亮 LED 灯教程中避开了寄存器映射, 采用对地址进行位操作

偏移地址

它的存在是历史原因所致

把存储单元的实际地址与其所在段的段地址之间的距离称为段内偏移，也称为“有效地址或偏移量”。亦：存储单元的实际地址与其所在段的段地址之间的距离。本质其实就是“实际地址与其所在段的段地址之间的距离”

更通俗一点讲，内存中存储数据的方式是：一个存储数据的“实际地址”=段首地址+偏移量，

也可以这样理解：就像我们现实中的“家庭地址”=“小区地址”+“门牌号”

上面的“偏移量”就好比“门牌号”

其实就相当于 C++的指针一样啦，指出确切的地址而已.....

汇编语言

[汇编语言基础知识](#)

Heap（堆）

因为用户主动请求而划分出来的内存区域，叫做 Heap（堆）。它由起始地址开始，从低位（地址）向高位（地址）增长。Heap 的一个重要特点就是不会自动消失，必须手动释放，或者由垃圾回收机制来回收。

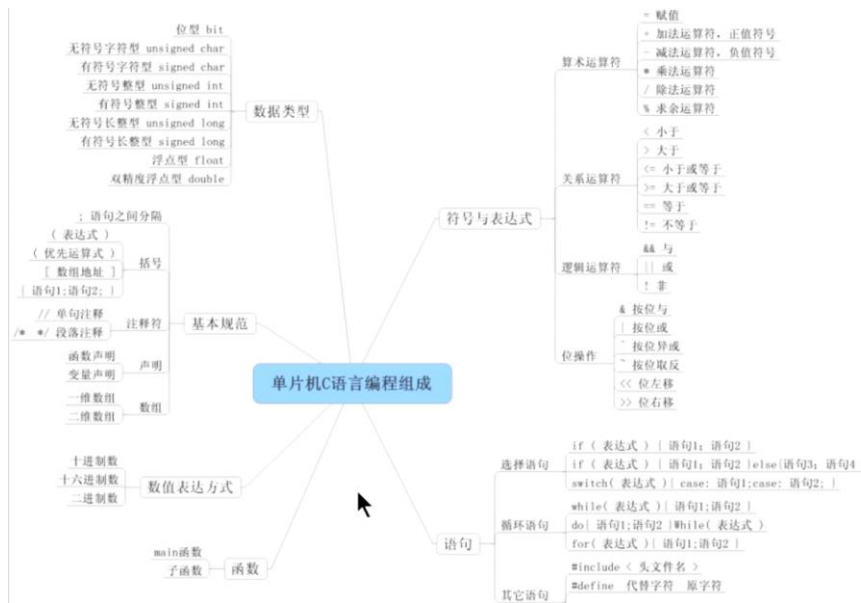
Stack（栈）

除了 Heap 以外，其他的内存占用叫做 Stack（栈）。简单说，Stack 是由于函数运行而临时占用的内存区域。

帧（frame）

帧是栈的一部分

C 语言



宏定义

条件编译

条件编译是一种宏定义，故有#，它的目的就是防止函数二次定义

最常用的方式就是

`#ifndef` //如果未定义此函数

`#define` //则定义它

续行符

语法：“\”

表示续行符的下一行与续行符所在的代码是连接起来

应用续行符的时候要注意，在“\”后面不能有任何字符(包括注释、空格)，只能直接

回车

指针

指针意思是通过它能找到以它为地址的内存单元

作个比喻，假设将电脑存储器当成一本书，一张内容记录了某个页码加上行号的便利贴，可以被当成是一个指向特定页面的指针

枚举

从同一类型的数据中抽出来一部分，就是枚举

Enum

<pre>enum 枚举名 { 标识符[=整型常数], 标识符[=整型常数], ... 标识符[=整型常数] } 枚举变量;</pre>	<pre>enum 通讯录 { 爸爸=139045388670122, 妈妈=151032137656790, 老婆=150222317649201, 老板=1398888888888888, 小张=152740387921233 } 拨出号码;</pre>	<pre>enum ABC { a=4, b=88, c=50, d=99 }x;</pre>
枚举变量 = 标识符;	拨出号码 = 小张;	x = b;
<pre>enum ABC { a, b, c, d }x;</pre>	<pre>enum ABC { a=5, b, c, d }x;</pre>	<pre>enum ABC { a, b, c=5, d }x;</pre>
a=0,b=1,c=2,d=3	a=5,b=6,c=7,d=8	a=0,b=1,c=5,d=6

第1个不赋值为0，其他不赋值是上一个值加1



结构体

把不同类型的数据重组在一块

枚举和结构体的区别

枚举是在一个数据类型中只选择一部分需要的数据。

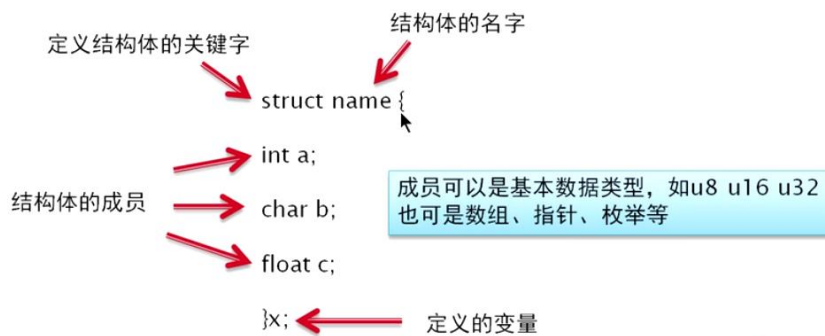


只需要一个类型中的分部数据

结构体是把多个不同类型的数据集合在一个类型之中。



需要多个数据类型



定义结构体	写入数据	数据调用
<pre>struct name { int a; char b; float c; }x;</pre> int 相当于 u16 char 相当于 u8	<pre>x.a = 65500; x.b = 255; x.c = 0x30;</pre> 其中 “.” 后面接成员名	<pre>if(x.a > 1){ z = x.b; }</pre>
定义结构体	写入数据	数据调用
<pre>typedef struct { int a; char b; float c; }x;</pre> x变成了一种数据类型 可用来定义变量	<pre>x y; y.a = 65500; y.b = 255; y.c = 0x30;</pre> 定义的变量y y的类型是x 这里的x类似于u8	<pre>if(y.a > 1){ z = y.b; }</pre>

typedef 为 C 语言的关键字，作用是作为一种数据类型定义一个新名字。这里的数据类型包括内部数据类型（int,char 等）和自定义的数据类型（struct 等）。

init 为初始化函数

变量

[浅析 C 语言之 uint8 t / uint16 t / uint32 t / uint64 t](#)

基本数据类型

数据类型	定义语句	占用空间	数值范围
位型	bit	1个位	0, 1
无符号字符型	unsigned char	1个字节	0~255
有符号字符型	signed char	1个字节	-128~127
无符号整型	unsigned int	2个字节	0~65535
有符号整型	signed int	2个字节	-32768~32767
无符号长整型	unsigned long	4个字节	0~4294967295
有符号长整型	signed long	4个字节	-2147483648~2147483647
浮点型	float	4个字节	$\pm 1.176E-38 \sim \pm 3.40E+38$ (6位)
双精度浮点型	double	8个字节	$\pm 1.176E-38 \sim \pm 3.40E+38$ (10位)

```
typedef unsigned long u32;          /* 无符号 32 位数 */
typedef unsigned short u16;         /* 无符号 16 位数 */
typedef unsigned char u8;           /* 无符号 8 位数 */
typedef unsigned long const uc32;   /* 无符号只读 32 位数 */
typedef unsigned short const uc16;  /* 无符号只读 16 位数 */
typedef unsigned char const uc8;    /* 无符号只读 8 位数 */
typedef volatile unsigned long vu32; /* volatile 修饰的无符号 32 位数 */
typedef volatile unsigned short vu16; /* volatile 修饰的无符号 16 位数 */
typedef volatile unsigned char vu8; /* volatile 修饰的无符号 8 位数 */
typedef volatile unsigned long const vuc32; /* volatile 修饰的无符号只读 32 位数 */
typedef volatile unsigned short const vuc16; /* volatile 修饰的无符号只读 16 位数 */
typedef volatile unsigned char const vuc8; /* volatile 修饰的无符号只读 8 位数 */
```

```

u32 a; //定义32位无符号变量a
u16 a; //定义16位无符号变量a
u8 a; //定义8位无符号变量a
vu32 a; //定义易变的32位无符号变量a
vu16 a; //定义易变的 16位无符号变量a
vu8 a; //定义易变的 8位无符号变量a
uc32 a; //定义只读的32位无符号变量a
uc16 a; //定义只读 的16位无符号变量a
uc8 a; //定义只读 的8位无符号变量a

```

可用格式:

```

u8 a;
u32 a,b,c;
u16 a = 0x0000;

```

数组

```

unsigned char code name [10]= //一维数组
{0x7F,0x02,0x0C,0x02,0x7F,0x3E,0x41,0x41,0x41,0x3E};

```

```

unsigned char code name [3][6]={ //二维数组
{0x7E,0xFF,0x81,0x81,0xFF,0x7E},
{0xC6,0xE7,0xB1,0x99,0x8F,0x86},
{0x42,0xC3,0x89,0x89,0xFF,0x76},
};

```

```

c = name[2]; //将一维数组中的第3个数值送入变量c中（数组从0开始读数）
c = name[1][2]; //将二维数组中的第2行的第3个数值送入变量c中

```

易变/修饰 只读变量

编译器有可能会对没有执行程序的变量进行优化

`volatile` 表示易变的变量，防止编译器优化

const：只读的变量。const变量的值在程序运行期间不能改变，不能再赋值。这种变量称为常量（constant variable）或是只读变量（read-only-variable）即存放在FLASH当中。在制作数据表之类的固定数据时要用这种类型。

STM32的C语言中没有类型8051单片机的位定义（bit a;）
可以用u8短字节变量代替。

参数与变量区别

参数也是变量。变量很多种，参数变量是其中一种。

普通变量是你自己初始化的，参数变量是程序自动为你初始化的（就在你调用函数的一瞬间）

追问

那带参函数是什么意思

它与不带参数的有什么区别

追答

从道理上讲，既然有 带参数，那就有不带参数的情况。

货车可以装货，也可以不装货啊，尽管货车是用来装货的。

从技术讲，定义一个函数实际上是定义了一段代码，一段可以重复利用的代码

这段代码只有入口和出口，就像工厂的生产线，你送进去棉花，出来就是毛线。

定义一个函数

```
int plus(int a, int b) { return a + b; }
```

定义一个函数

成品 生产(原料 棉花) <--- 成品代表int 生产单表plus 原料代表int 棉花代表 a。

```
{  
    return （棉花在生产线上经过七七四十九关处理变成一一毛线）；  
}
```

```
}
```

有时候函数没有参数，这个就跟自然数为什么要发明一个 0 呢，一样道理。

函数这段代码有一个专门传入参数的地方，如果没有参数，这个地方就没用而已。

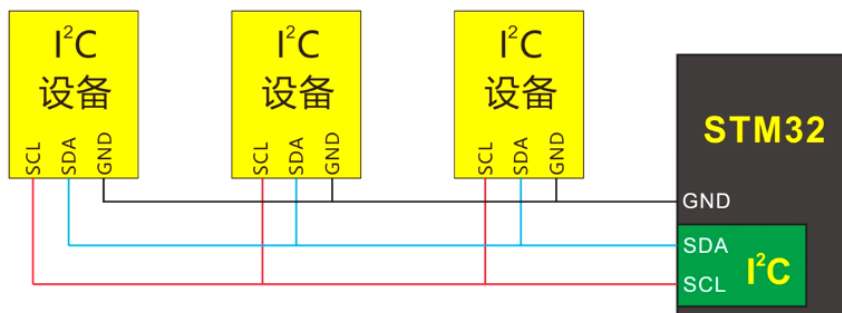
[C 语言参数和变量的区别](#)

[C 语言形参和实参的区别（非常详细）](#)

简单来说，形参就是无赋值，而实参就是有赋值

总线

I2C 总线



- ✓ I2C总线是板级总线，连接线一般不超2米
- ✓ I2C的数据线上理论上需要加2K的上拉电阻
- ✓ 所有设备与单片机需要共地

SCL 时钟控制线 SDA 串行数据线

Serial clock data

AHB 总线

AHB 总线衍生出 APB2 和 APB1 总线，上面挂载着 STM32 各种各样的特色外设

2.3.7 时钟和启动

系统时钟的选择是在启动时进行，复位时内部8MHz的RC振荡器被选为默认的CPU时钟，随后可以选择外部的、具失效监控的4~16MHz时钟；当检测到外部时钟失效时，它将被隔离，系统将自动地切换到内部的RC振荡器，如果使能了中断，软件可以接收到相应的中断。同样，在需要时可以采取对PLL时钟完全的中断管理(如当一个间接使用的外部振荡器失效时)。

多个预分频器用于配置AHB的频率、高速APB(APB2)和低速APB(APB1)区域。AHB和高速APB的最高频率是72MHz，低速APB的最高频率为36MHz。参考图2的时钟驱动框图。

- ✓ AHB是高级高性能总线，用于CPU、DMA、DSP的通信
- ✓ APB是外围总线，用于内部其他功能的通信
- ✓ APB分为高速APB2和低速APB1

固件库

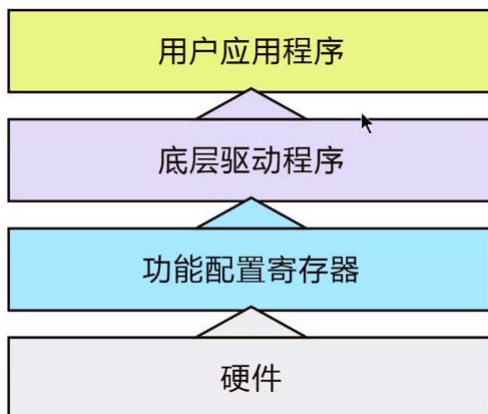
中英文对照

Firmware library

模板：template

bsp：Board support package 板级支持包；就是只针对特定的开发板

为什么用固件库



固件库就是底层驱动程序，用来操作寄存器

	易学	易用	移植性	占用空间	完善程度	执行效率	硬件覆盖范围
寄存器操作	差	差	差	小	差	高	小
标准固件库	中	中	中	中	中	中	中
STM32Cube (HAL)	优	优	优	大	优	低	大
STM32Cube (LL)	优	差	差	小	中	高	小

Doxygen

```
/**
 * @brief 初始化控制 LED 的 IO
 * @param 无
 * @retval 无
 */
```

这一段是代码书写的规范，名为“Doxygen”，如果在工程文件中按照这种规范去注释，可以使用 Doxygen 软件自动根据注释生成帮助文档。

函数简介(@brief)、参数说明('@param')和返回值说明(@retval)四部分

文件类型

.s 是汇编语言

.h 是头文件

.h 中一般放的是宏定义以及同名.c 文件中定义的变量、数组、函数的声明，需要让.c 外部使用的声明。

.c 文件一般放的是变量、数组、函数的具体定义。

用法

.c 文件，以 c 为扩展名，一般存储具体功能的实现。

.h 文件，称为头文件，一般存储类型的定义（宏定义），函数的声明等。通常，头文件被.c 文件包含，使用#include 语句。但值得注意的是，这只是一种约定，而非强制。

Ps：为防止.h 被不同.c 文件调用而导致其中宏定义被多次定义【类似一个函数可以被声明

多次，但是只能被定义一次】，在.h文件中宏定义时一定要用条件编译

文件作用

1-汇编编写的启动文件

startup_stm32f10x_hd.s: 设置堆栈指针、设置 PC 指针、初始化中断向量表、配置系统时钟、使用库函数_main 最终去到 C 的世界

2-时钟配置文件

system_stm32f10x.c: 把外部时钟 HSE=8M，经过 PLL 倍频为 72M。

3-外设相关的

stm32f10x.h: 实现了内核之外的外设的寄存器映射

本质还是头文件

但是一般来说由于功能重要，故在 main.c 中必须要定义

xxx: GPIO、USART、I2C、SPI、FSMC

stm32f10x_xx.c: 外设的驱动函数库文件

stm32f10x_xx.h: 存放外设的初始化结构体，外设初始化结构体成员的参数列表，外设固件库函数的声明

4-内核相关的

CMSIS - Cortex 微控制器软件接口标准

core_cm3.h: 实现了内核里面外设的寄存器映射

core_cm3.c: 内核外设的驱动固件库

NVIC(嵌套向量中断控制器)、SysTick(系统滴答定时器)

misc.h

misc.c

5-头文件的配置文件

stm32f10x_conf.h: 头文件的头文件

//stm32f10x_usart.h

//stm32f10x_i2c.h

//stm32f10x_spi.h

//stm32f10x_adc.h

//stm32f10x_fsmc.h

.....

6-专门存放中断服务函数的 C 文件

stm32f10x_it.c

stm32f10x_it.h

中断服务函数可以随意放在其他的地方，并不是一定要放在 stm32f10x_it.c

固件库调用逻辑

STARTUP 文件启动[汇编语言，效率高]

调用 CMSIS 中的 core_cm3.c[ARM 内核相关程序]以及 system_stm32f10x.c 来配置时钟

使用 FWLib 添加固件库来关联 main.c

在头文件 main.c 中对固件库进行调用

Ps:在 main.c 前声明 stm32f10x.h 就可以调用这个库文件[其中对所有功能配置寄存器的地址进行了定义，之后就可以通过定义名来操作地址]

由于我们基本的函数执行均在 main.c 中完成，所以编写头文件的过程一般称作初始化 xxx，因为头文件里面的内容是不需要全部调用的，而.c 文件中如果有变量定义了而未进行使用，

则会 warning

野火新建工程文件存放方式

Fwlib-Template\Libraries\CMSIS\startup

Fwlib-Template\Libraries\CMSIS\core_cm3.c+stm32f10x.c

Fwlib-Template\Libraries\STM32F10x_StdPeriph_Driver\inc(库.h)+src(函数.c)

Fwlib-Template\User\main.c+stm32f103_it.c

洋桃新建工程文件存放方式

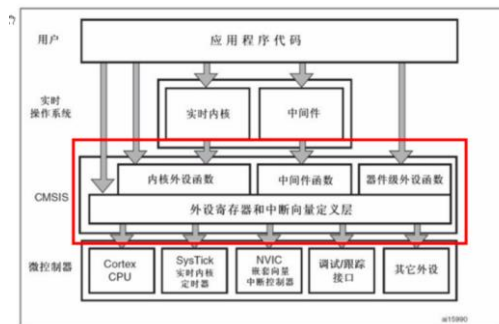
CMSIS\core_cm3.c+stm32f10x.c

Lib\STM32F10x_StdPeriph_Driver\inc(库.h)+src(函数.c)

startup

user\main.c+stm32f103_it.c

CMSIS：微控制器软件接口标准



GPIO

GPIO 其实是厂商的说辞，本质还是 I/O 端口

2.3.21 通用输入输出接口(GPIO)

每个GPIO引脚都可以由软件配置成输出(推挽或开漏)、输入(带或不带上拉或下拉)或复用的外设功能端口。多数GPIO引脚都与数字或模拟的复用外设共用。除了具有模拟输入功能的端口，所有的GPIO引脚都有大电流通过能力。

在需要的情况下，I/O引脚的外设功能可以通过一个特定的操作锁定，以避免意外的写入I/O寄存器。
在APB2上的I/O脚可达18MHz的翻转速度。

翻转 toggle

中英文对照

PWM 脉冲宽度调制

LSB，英文 least significant bit，中文义最低有效位

Alias，别名

引脚与 I/O 端口

I/O 端口是引脚的一大类使用方式

引脚有 3 个要注意的地方：

1. 大部分引脚也会有复用
2. 少部分引脚可以自己映射（重定义）
3. 大部分引脚兼容 5V

GPIO 中所用到的寄存器

CR:配置寄存器

DR: 数据寄存器

每个 GPI/O 端口有两个 32 位配置寄存器(GPIOx_CRL, GPIOx_CRH): LOW 代表低电平, HIGH 代表高电平 **方向控制**

两个 32 位数据寄存器 (GPIOx_IDR 和 GPIOx_ODR): input output **电平控制**

一个 32 位置位/复位寄存器(GPIOx_BSRR): B 代表 bit set reset

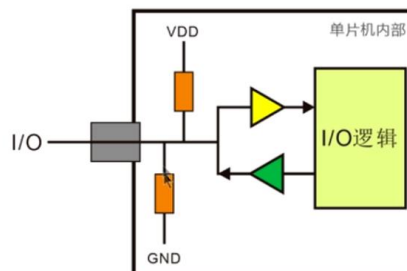
一个 16 位复位寄存器(GPIOx_BRR)

一个 32 位锁定寄存器(GPIOx_LCKR):lock

GPIO 的 8 种输入方式

翻转速度指的是方波脉冲翻转速度

GPIO_Mode_AIN 模拟输入
GPIO_Mode_IN_FLOATING 浮空输入
GPIO_Mode_IPD 下拉输入
GPIO_Mode_IPU 上拉输入
GPIO_Mode_Out_PP 推挽输出
GPIO_Mode_Out_OD 开漏输出
GPIO_Mode_AF_PP 复用推挽输出
GPIO_Mode_AF_OD 复用开漏输出



模拟输入即 ADC 输入

模拟和浮空都是断开上下拉电阻和输出端（绿色放大器），只有输入（黄色放大器）只不过输入信号不同

上拉/下拉的目的是保持端口悬空时的电平状态【上拉/下拉的电阻很大，故不会影响实际输入高/低电平】

推挽电流较大，有驱动外设工作的能力

开漏电流较小，只是一个输出电流

GPIO 的分组

C51 中以 1.1, 1.2; 3.1, 3.2 这样的方式分组

STM32 中以 PA,PB,PC 这样的方式分组，且并非所有端口都会列出来（有一些是内部自己使用了）

GPIO 初始化配置函数解析

[stm32 GPIO 简单介绍及初始化配置（库函数）](#)

延时函数代码解析

```
1.void Delay(__IO uint32_t nCount)    //简单的延时函数
{
    for(; nCount != 0; nCount--);

}

//具体时间与#define SOFT_DELAY Delay(0xFFFFF)以及时钟频率有关
```

初始化配置代码解析

定义一个固定类型的结构体
开启相关时钟
选择控制引脚
设置引脚mode,speed
把设置好的参数初始化到外设中

```
1 void LED_GPIO_Config(void)
2 {
3 /*定义一个 GPIO_InitTypeDef 类型的结构体*/
4 GPIO_InitTypeDef GPIO_InitStructure;
5
6 /*开启 LED 相关的 GPIO 外设时钟*/
7 RCC_APB2PeriphClockCmd( LED1_GPIO_CLK|
8                          LED2_GPIO_CLK|
9                          LED3_GPIO_CLK, ENABLE);
10 /*选择要控制的 GPIO 引脚*/
11 GPIO_InitStructure.GPIO_Pin = LED1_GPIO_PIN;
12
13 /*设置引脚模式为通用推挽输出*/
14 GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
15
16 /*设置引脚速率为 50MHz */
17 GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

批注 [1]: 如果把函数比喻成一台机器，那么参数就是原材料，返回值就是最终产品；从一定程度上讲，函数的作用就是根据不同的参数产生不同的返回值。
这里的 nCount 就是无符号 32 位形参

批注 [2]: For 后面表达式 1 的初始值已经被定义了
!= 意思是不等于
这就是个递减

批注 [3]: 无返回值无参数主函数 LED_GPIO_Config

批注 [4]: C 语言规定，变量的定义，必须位于函数的前面（用于定义全局变量），或者在函数内部也必须放于可执行语句的前部。不允许出现先有代码再定义变量之后又有代码的混合结构。
所以 GPIO_InitTypeDef 是一个结构体类型，GPIO_InitStructure 是这个结构体名称。

批注 [5]: RCC_APB2PeriphClockCmd () 是一个函数调用，是具体的可执行代码。
是操作寄存器按位或

批注 [6]: Struct.a 格式正是结构体的调用

```

18
19 /*调用库函数，初始化 GPIO*/
20 GPIO_Init(LED1_GPIO_PORT, &GPIO_InitStructure);
21
22 /*选择要控制的 GPIO 引脚*/
23 GPIO_InitStructure.GPIO_Pin = LED2_GPIO_PIN;
24
25 /*调用库函数，初始化 GPIO*/
26 GPIO_Init(LED2_GPIO_PORT, &GPIO_InitStructure);
27
28 /*选择要控制的 GPIO 引脚*/
29 GPIO_InitStructure.GPIO_Pin = LED3_GPIO_PIN;
30
31 /*调用库函数，初始化 GPIO*/
32 GPIO_Init(LED3_GPIO_PORT, &GPIO_InitStructure);
33
34 /* 关闭所有 led 灯 */
35 GPIO_SetBits(LED1_GPIO_PORT, LED1_GPIO_PIN);
36
37 /* 关闭所有 led 灯 */
38 GPIO_SetBits(LED2_GPIO_PORT, LED2_GPIO_PIN);
39
40 /* 关闭所有 led 灯 */
41 GPIO_SetBits(LED3_GPIO_PORT, LED3_GPIO_PIN);
42 }

```

附件：上述代码原图

批注 [7]: 调用外设初始化函数，初始化函数也是在官方库函数的对应头文件里。比如这里找 GPIO 函数则应该在“stm32f10x_gpio.h”里。符号“&”是取地址符，意思是：初始化的参数地址为。。。 (然后 MDK 就寻找结构体的位置，以找到结构体的参数)。所以这里调用的方式可以总结为将事先设定好的参数 (在 GPIO_InitStructure 中进行设置) 告诉要用到的外设 (LED1_GPIO_PORT): 也可以说是初始化 LED1_GPIO_PORT 的参数为以上结构体 GPIO_InitStructure

批注 [8]: Setbits 输出高电平
Resetbits 输出低电平
这里灯的开关是负逻辑

```

1 void LED_GPIO_Config(void)
2 {
3     /*定义一个 GPIO_InitTypeDef 类型的结构体*/
4     GPIO_InitTypeDef GPIO_InitStructure;
5
6     /*开启 LED 相关的 GPIO 外设时钟*/
7     RCC_APB2PeriphClockCmd( LED1_GPIO_CLK|
8                             LED2_GPIO_CLK|
9                             LED3_GPIO_CLK, ENABLE);
10
11    /*选择要控制的 GPIO 引脚*/
12    GPIO_InitStructure.GPIO_Pin = LED1_GPIO_PIN;
13
14    /*设置引脚模式为通用推挽输出*/
15    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
16
17    /*设置引脚速率为 50MHz */
18    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
19
20    /*调用库函数，初始化 GPIO*/
21    GPIO_Init(LED1_GPIO_PORT, &GPIO_InitStructure);
22
23    /*选择要控制的 GPIO 引脚*/
24    GPIO_InitStructure.GPIO_Pin = LED2_GPIO_PIN;
25
26    /*调用库函数，初始化 GPIO*/
27    GPIO_Init(LED2_GPIO_PORT, &GPIO_InitStructure);
28
29    /*选择要控制的 GPIO 引脚*/
30    GPIO_InitStructure.GPIO_Pin = LED3_GPIO_PIN;
31
32    /*调用库函数，初始化 GPIO*/
33    GPIO_Init(LED3_GPIO_PORT, &GPIO_InitStructure);
34
35    /* 关闭所有 led 灯 */
36    GPIO_SetBits(LED1_GPIO_PORT, LED1_GPIO_PIN);
37
38    /* 关闭所有 led 灯 */
39    GPIO_SetBits(LED2_GPIO_PORT, LED2_GPIO_PIN);
40
41    /* 关闭所有 led 灯 */
42    GPIO_SetBits(LED3_GPIO_PORT, LED3_GPIO_PIN);
43 }

```

位带操作(偏重寄存器，可忽略)

计算机位的理解

我们常常看到 32 位 CPU、64 位 CPU 这样的名称，其实指的就是寄存器的大小

32 位是说计算机一次最多能够处理 32 位数据，1byte=8bit，也就是说 32 位就是 4 字节

不同位支持的运行内存不同。

每个存储单元可以存放八个二进制位，即一个零到二百五十五之间的整数、一个字母或一个标点符号等，叫做一个字节），即 1 字节=8 位。**存储器的容量就是以字节为基本单位的，每个单元都有唯一的序号，叫做地址。**中央处理器凭借地址，准确地操纵着每个单元，处理数据。

Eg:比如外设外带区的地址为：0X40000000~0X40100000，大小就为 1MB

因为 1 在第 6 位，即 $16^5=2^{20}$,单位 byte,1MB=2¹⁰KB,1KB=2¹⁰byte

指针的寻址能力是 4byte，段地址+偏移地址，可以通过一个地址找到整个寄存器

但是一个地址仍然对应着的是一个字节

参考手册原文：

必须以字(32位)的方式操作这些外设寄存器。

寻址本身也是一种操作，所以寻址时时必须对整个 32 位寄存器进行操作

位操作和位带操作

位操作就是可以单独的对一个比特位【地址最末位】读和写

51 单片机中通过关键字 sbit 来实现位定义

51 单片机里面并不是所有的寄存器都是可以比特位操作，有些寄存器还是得字节操作，比如 SBUF

STM32 没有这样的关键字，但是通过访问位带别名区可以实现对所有寄存器位操作

在 STM32 中，有两个特定的地方，一个是 SRAM 区的最低 1MB 空间，另一个是外设区最低 1MB 空间，称为位带，位带经过膨胀后得到位带别名区

对位带别名区的操作称为位带操作

RCC

中英文对照

RCC : reset clock control **复位和时钟**控制器【注意该控制器有时钟和复位两种功能】

高速外部时钟信号 HSE(High Speed **External** Clock signal)相应的还有 HSI(High Speed **Internal** Clock signal)、LSE(Low..)、LSI(Low..)

RCC 寄存器英文缩写均省略 clock

时钟控制寄存器 CR : control register

时钟配置寄存器 CFGR : configuration register

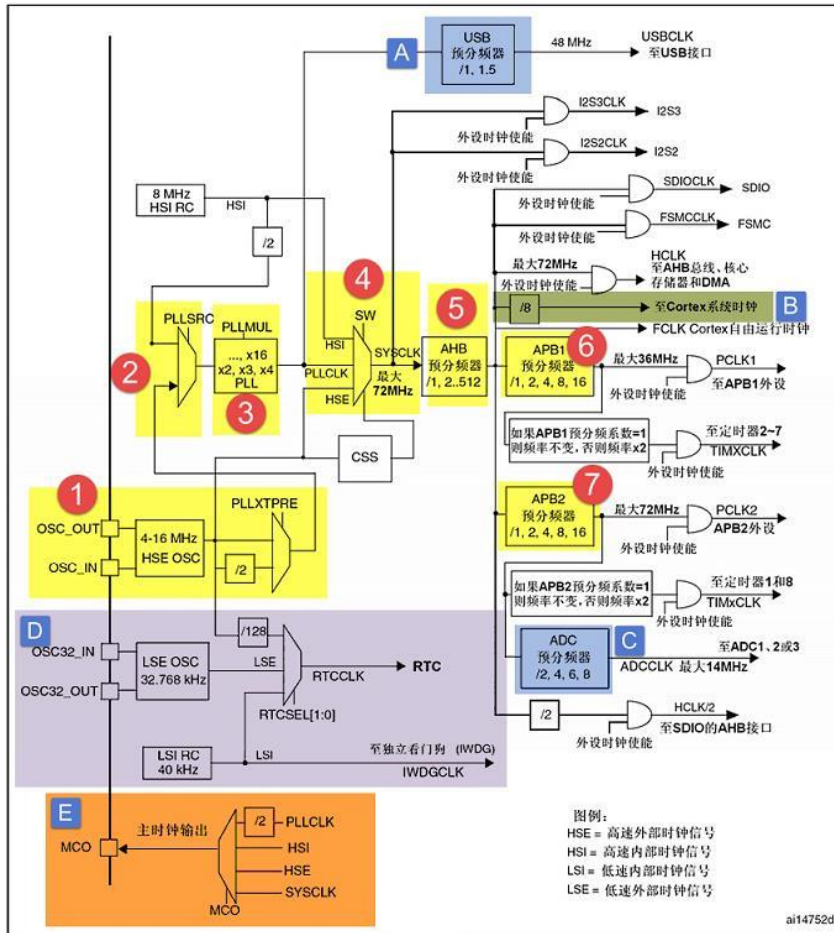
1. PLLMUL: PLL 倍频系数 (PLL multiplication factor)
2. PLLXTPRE: HSE 分频器作为 PLL 输入 (HSE divider for PLL entry)**PRE 想说的是预分频**
3. PLLSRC: PLL 输入时钟源 (PLL entry clock source)

备份域控制寄存器 BDCR:Backup Domain Control Register

控制/状态寄存器 CSR:control/status register

时钟

时钟树



对时钟的理解

STM32 中的高速时钟是给内核和外设用的，而低速时钟是给 RTC 和 IWDG 用的

高速单位一般是 MHz，低速是 KHz

HSE 8M 倍频成 72M，当 HSE 不能用时，使用 HSI 时钟

E 是 external，I 是 internal，外部始终不能用才用内部，因为内部时钟会产生温漂

一个时钟可以通过分频产生不同的频率，频率输送到定时器开始计数，实现精准计时

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留					MCO[2:0]			保留	USB PRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
					rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADCPRE[1:0]		PPRE2[2:0]			PPRE1[2:0]			HPRE[3:0]			SWS[1:0]		SW[1:0]		

分频和倍频

CPU 时钟频率称为主频，主频频率高，所以需要倍频得到其时钟信号

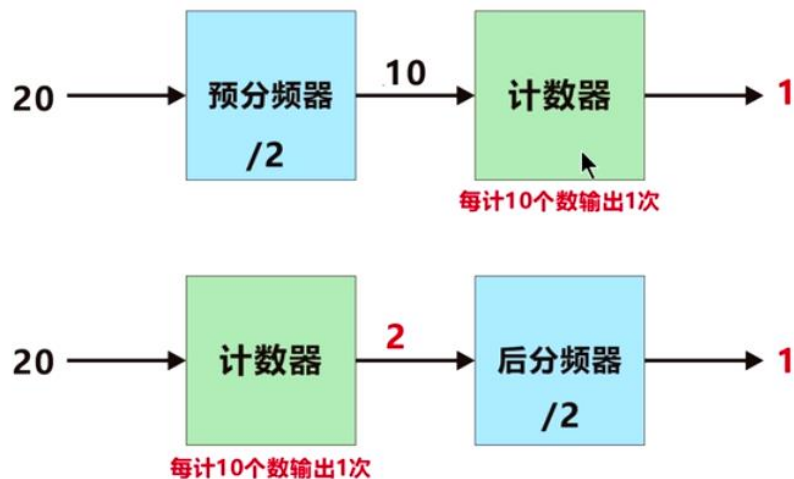
其他外设频率低，且所需频率不一，所以需要分频

倍频



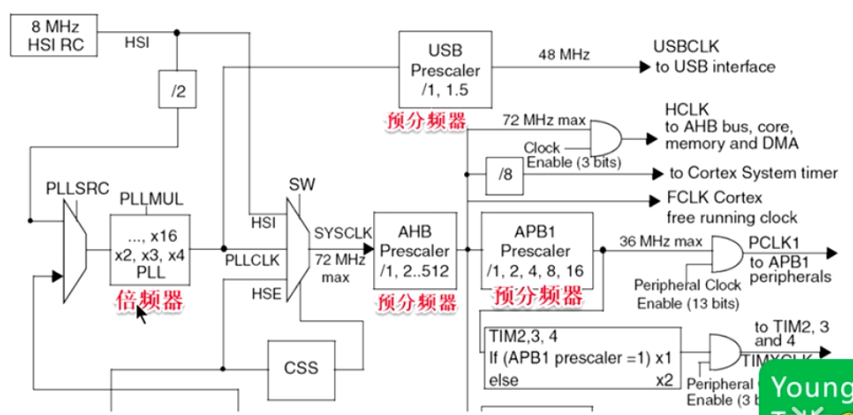
- ✓ 倍频是输入频率的翻倍
- ✓ PLL是指锁相环电路
- ✓ 锁相环电路的功能是加倍频率
- ✓ 所以用PLL代指倍频器

预分频和后分频



- ✓ 预分频是让计数器输入值减少几分之几
- ✓ 后分频是让计数器输出次数减少几分之几

后分频几乎不用，预分频英文 **prescaler**



晶振

分类：分为有源晶振和无源晶振

有源晶振需要电源，但不需要附加的电容。它内部含有起振器。通常接到 CPU 的一根引脚即可【OSC_IN】。无源晶振不需要电源，但常需要和电容配合使用，它通常接到 CPU 的两根引脚上。【OSC_IN 和 OSC_OUT】

目的：产生时钟信号（一般是方波）

系统时钟

软件延时函数，使用不同的系统时钟，延时时间不一样，可以通过 LED 闪烁的频率来判断。

其他时钟

MCO 是 microcontroller clock output 的缩写，是微控制器时钟输出引脚，在 STM32 F1 系列中 由 PA8 复用所得，主要作用是可以对外提供时钟，**相当于一个有源晶振可以用来监控系统时钟**

中断

STM32 具有十分强大的中断系统，将中断分为了两个类型：系统异常和外部中断。并将所有中断通过一个表编排起来，称之为 stm32 中断向量表

系统异常体现在内核水平，故也叫内核异常

外部中断体现在外设水平

内核异常不能够被打断，不能被设置优先级（也就是说优先级是凌驾于外部中断之上的）。常见的内核异常有以下几种：复位(reset),不可屏蔽中断(NMI),硬错误(Hardfault)，其他的也可以在表上找到。

外部中断包含线中断，定时器中断，I2C，SPI 等所有的外设中断，可配置优先级

接下来讨论的都是外部中断

一般情况下中断和异常可以混用，不做区分

注意中断一定是终止内核的工作

中英文对照

Interrupt

异常服务例程(ESR)：Exception service routine

系统控制块(System Control Block)

应用程序中断及复位控制寄存器 AIRCR: Application Interrupt and Reset Control register

PRIGROUP: PriorityGroup

API (Application Programming Interface, 应用程序接口): 是一些预先定义的函数

对中断的理解

有关单片机中断系统的概念: 什么是中断, 我们从一个生活中的例程引入。你正在家中看书, 突然电话铃响了, 你放下书本, 去接电话, 和来电话的人交谈, 然后放下电话, 回来继续看你的书。这就是生活中的“中断”的现象, 就是正常的工作过程被外部的事件打断了。仔细研究一下生活中的中断, 对于我们学习单片机的中断也很有好处。

第一、什么可经引起中断, 生活中很多事件能引起中断: 有人按了门铃了, 电话铃响了, 你的闹钟闹响了, 你烧的水开了...等等诸如此类的事件, 我们把能引起中断的称之为中断源。

第二、中断的嵌套与优先级处理: 设想一下, 我们正在看书, 电话铃响了, 同时又有人按了门铃, 你该先做那样呢? 如果你正是在等一个很重要的电话, 你一般不会去理会门铃的, 而反之, 你正在等一个重要的客人, 则可能就不会去理会电话了。如果不是这两者(即不等电话, 也不是等人上门), 你可能会按你常常的习惯去处理。总之这里存在一个优先级的问題, 单片机中也是如此, 也有优先级的问題。优先级的问題不仅仅发生在两个中断同时产生的情况, 也发生在一个中断已产生, 又有一个中断产生的情况, 比如你正接电话, 有人按门铃的情况, 或你正开门与人交谈, 又有电话响了情况。考虑一下我们会怎么办吧。

第三、中断的响应过程: 当有事件产生, 进入中断之前我们必须先记住现在看书的第几页了, 或拿一个书签放在当前页的位置, 然后去处理不一样的事情(因为处理完了, 我们还要回来继续看书): 电话铃响我们要到放电话的地方去, 门铃响我们要到门那边去, 也说是不一样的中断, 我们要在不一样的地点处理, 而这个地点常常还是固定的。计算机中也是采用的这种办法, 五个中断源, 每个中断产生后都到一个固定的地方去找处理这个中断的程序, 当然在去之前首先要保存下面将执行的指令的地址, 以便处理完中断后回到原来的地方继续往下执行程序。具体地说, 中断响应能分为以下几个步骤:

- 1、保护断点, 即保存下一将要执行的指令的地址, 就是把这个地址送入堆栈。
- 2、寻找中断入口, 根据不一样的中断源所产生的中断, 查找不一样的入口地址。
- 3、执行中断处理程序。

4、中断返回：执行完中断指令后，就从中断处返回到主程序，继续执行。

NVIC

NVIC 是主要的中断控制器，外部内部中断均能处理。它跟内核紧密耦合，是内核里面的一个外设。但是各个芯片厂商在设计芯片的时候会对 Cortex-M3 内核里面的 NVIC 进行裁剪，把不需要的部分去掉，所以说 STM32 的 NVIC 是 Cortex-M3 的 NVIC 的一个子集。

NVIC 负责除了 SYSTICK 之外的所有中断的控制，十分重要！

优先级

分类

分成抢占优先级和子优先级。如果有多个中断同时响应，抢占优先级高的就会抢占 抢占优先级低的 优先得到执行，如果抢占优先级相同，就比较子优先级。如果抢占优先级和子优先级都相同的话，就比较他们的硬件中断编号，编号越小，优先级越高

分组

优先级分组	主优先级	子优先级	描述
NVIC_PriorityGroup_0	0	0-15	主-0bit, 子-4bit
NVIC_PriorityGroup_1	0-1	0-7	主-1bit, 子-3bit
NVIC_PriorityGroup_2	0-3	0-3	主-2bit, 子-2bit
NVIC_PriorityGroup_3	0-7	0-1	主-3bit, 子-1bit
NVIC_PriorityGroup_4	0-15	0	主-4bit, 子-0bit

第 0 组：所有的 4 位都有来表示响应优先级，能够配置 16 种不同的响应优先级。中断优先级则都相同。

第 1 组：最高一位用来配置抢占优先级，剩余三位用来表示响应优先级。那么就有两种不同的抢占优先级(0 和 1)和 8 种不同的响应优先级(0~7)。

第 2 组：高两位用来配置抢占优先级，低位用来配置响应优先级。那么两种优先级就各有 4 种。

第 3 组：高三位用来配置抢占优先级，低位用来配置响应优先级。有 8 种抢占优先级和 2 种相应优先级。

第 4 组：所有位都用来配置抢占优先级，即有 16 种抢占优先级，没有响应属性

这 5 种不同的分配方式，根据项目的实际需求来配置。

配置的 API 如下：

`NVIC_PriorityGroupConfig();`

其中括号内可以输入以下一个参数，代表不同的分配方式：

`NVIC_PriorityGroup_0`

`NVIC_PriorityGroup_1`

`NVIC_PriorityGroup_2`

`NVIC_PriorityGroup_3`

`NVIC_PriorityGroup_4`

初始化 NVIC

`NVIC_IRQChannel` 需要配置的中断向量

`NVIC_IRQChannelCmd` 使能或者关闭相应中断向量的中断响应

`NVIC_IRQChannelPreemptionPriority` 配置相应中断向量的抢占优先级

`NVIC_IRQChannelSubPriority` 配置相应中断的响应优先级

中断编程

`misc` 是大杂烩的意思，`misc.c` 文件中放的就是一些其他文件没有写的东西，其中最重要的就是 `nvic` 的相关定义

同时 `core_cm3.c` 文件中也有 `nvic` 的相关定义

在中断编程中，我们一般不用 `core_cm3.h` 中固件库函数，而是采取下面的方法

1.使能中断请求

这个具体由每个外设的相关中断使能位控制

2.初始化 `NVIC_InitTypeDef` 结构体，配置中断优先级分组，设置抢占优先级和子优先级，使能中断请求

3、编写中断服务函数

为了方便管理我们把中断服务函数统一写在 `stm32f10x_it.c` 这个库文件中

AFIO

stm32 的引脚有两种用途：GPIO（general purpose io 通用功能）和 AFIO（alternate function io 复用功能）

对于一些引脚（视芯片而定），这两种用途都没有，如在 64 脚产品中，OSC_IN/OSC_OUT 与作为 GPIO 端口的 PD0/PD1 共用一样的引脚，而在 100、144 引脚产品中，这四个功能各有引脚与之对应，不互相冲突，所以 OSC_IN/OSC_OUT 既不作 GPIO 也不作 AFIO

一文看懂 stm32 的引脚的两种用途：GPIO 和 AFIO

EXTI

EXTI（External interrupt/event controller）—外部中断/事件控制器

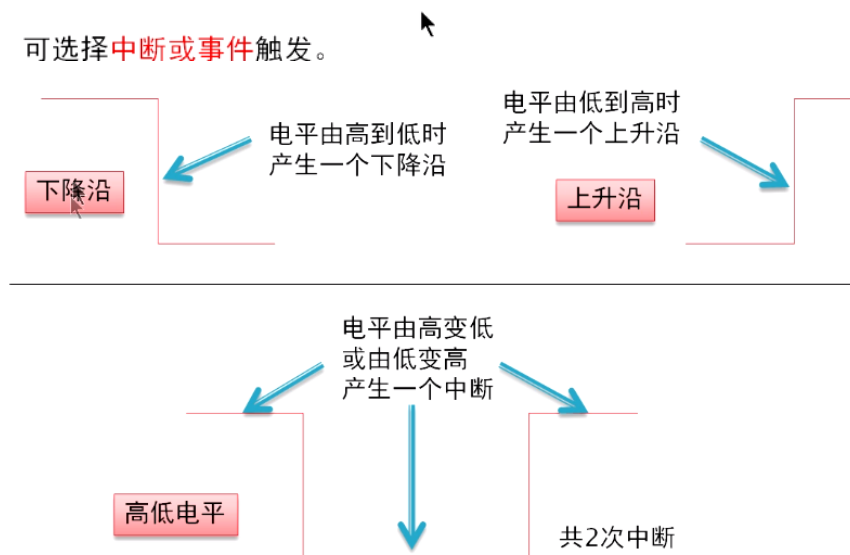
与 NVIC 不同，EXTI 仅处理外部

触发方式

STM32F1 支持将**所有GPIO**设置为中断输入。

外部IO可由**上沿**、**下沿**、**高低电平**的三种方式触发。

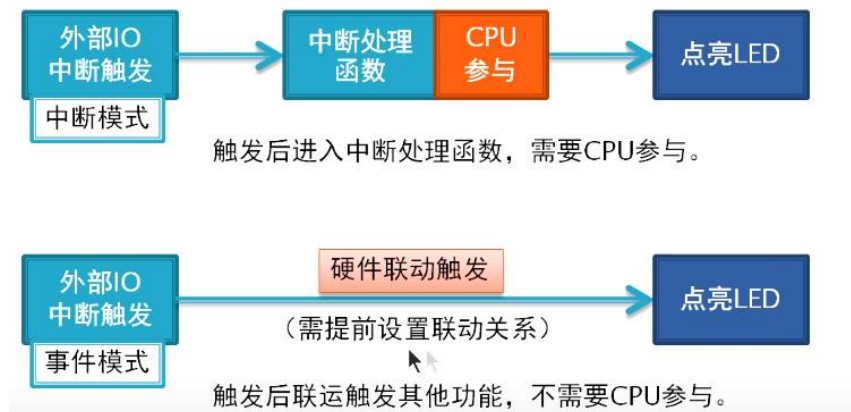
可选择**中断或事件**触发。



只有上升沿触发、只有下降沿触发或者上升沿和下降沿都触发

中断与事件

中断与事件



输入

EXTI 控制器有 19 个中断/事件输入线【第 20 个是以太网唤醒事件（只适用互联型）

互联型指的是 F105，107 等系列】，其中，EXTI0 至 EXTI15 用于 GPIO

当中断被触发后，程序要马上跳转到中断处理函数去执行中断操作，这个函数在工程创建时默认是没有的，需要手动添加。这个函数在 `startup_stm32f10x_hd.s` 中

GPIO引脚	中断标志位	中断处理函数
PA0~PG0	EXTI0	EXTI0_IRQHandler
PA1~PG1	EXTI1	EXTI1_IRQHandler
PA2~PG2	EXTI2	EXTI2_IRQHandler
PA3~PG3	EXTI3	EXTI3_IRQHandler
PA4~PG4	EXTI4	EXTI4_IRQHandler
PA5~PG5	EXTI5	EXTI9_5_IRQHandler
PA6~PG6	EXTI6	
PA7~PG7	EXTI7	
PA8~PG8	EXTI8	
PA9~PG9	EXTI9	
PA10~PG10	EXTI10	EXTI15_10_IRQHandler
PA11~PG11	EXTI11	
PA12~PG12	EXTI12	
PA13~PG13	EXTI13	
PA14~PG14	EXTI14	
PA15~PG15	EXTI15	

线 16：连接到 PVD 输出。
线 17：连接到 RTC 闹钟事件。
线 18：连接到 USB 唤醒事件。

EXTI 寄存器

AFIO_EXTICR 寄存器组，总共有 4 个，因为编译器的寄存器组都是从 0 开始编号的，所以 EXTI0[0]~EXTICR[3]，对应《STM32 参考手册》里的 EXTICR1~EXTICR 4。每个 EXTICR 只用了其低 16 位。

编程

- 1-初始化要连接到 EXTI 的 GPIO
- 2-初始化 EXTI 用于产生中断/事件
- 3-初始化 NVIC，用于处理中断
- 4-编写中断服务函数
- 5-main 函数

SYSTICK:嘀嗒定时器

中英文对照

Tick: 发出嘀嗒声

系统时基定时器 嘀嗒定时器

这个定时器是专用于**实时操作系统**，也可当成一个标准的递减计数器。它具有下述特性：

- ✓24位的递减计数器
- ✓自动重加载功能
- ✓当计数器为0时能产生一个可屏蔽系统中断
- ✓**可编程时钟源**

DMA

中英文对照

Memory:存储器 **Peripheral:**外设 **Configuration:** 配置

SRC: (source): 源头 DST (destination): 目的地

DIR: (direction): 方向

DMA_CPARx (DMA channel x-Peripheral address register): DMA 通道 x 外设地址寄存器(x=1...7)

理解为 DMA 的第 x 通道的外设地址寄存器

同理

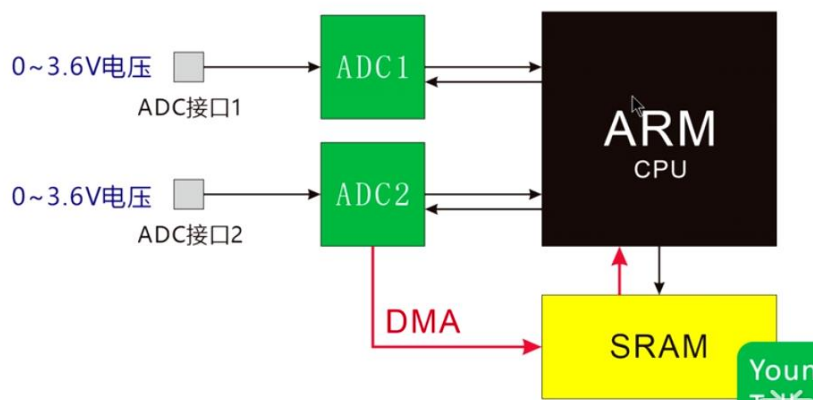
DMA_CCARx(DMA channel x-Configuration address register): DMA 通道 x 配置地址寄存器 (x=1...7)

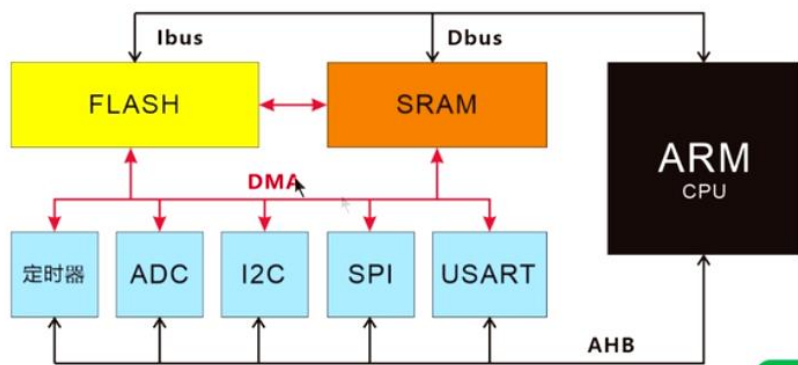
PINC: 外设地址增量模式 (Peripheral increment mode)

DMA_CNDTRx: (DMA channel x-Number of data to transfer register)DMA 通道 x (数据) 传输数量寄存器(x = 1...7)

DMA 配置

DMA(Direct Memory Access,直接存储器访问)用来提供在外设【Peripheral】和存储器之间或者存储器和存储器之间的高速数据传输。无须 CPU 干预，数据可以通过 DMA 快速地移动，这就节省了 CPU 的资源来做其他操作。





DMA功能原理示意图

数据分为常量和变量，常量放在 FLASH 中，CPU 通过 ICode 总线读取

变量放在 SRAM 中，CPU 通过 DCode 总线读取

数据还可被 DMA 总线读取

取数时经过总线矩阵仲裁，决定哪个总线读取

同时，不同外设之间还可以通过 DMA 互相读取数据

[DMA 配置方法 1](#)

[DMA 配置方法 2](#)

ADC

中英文对照

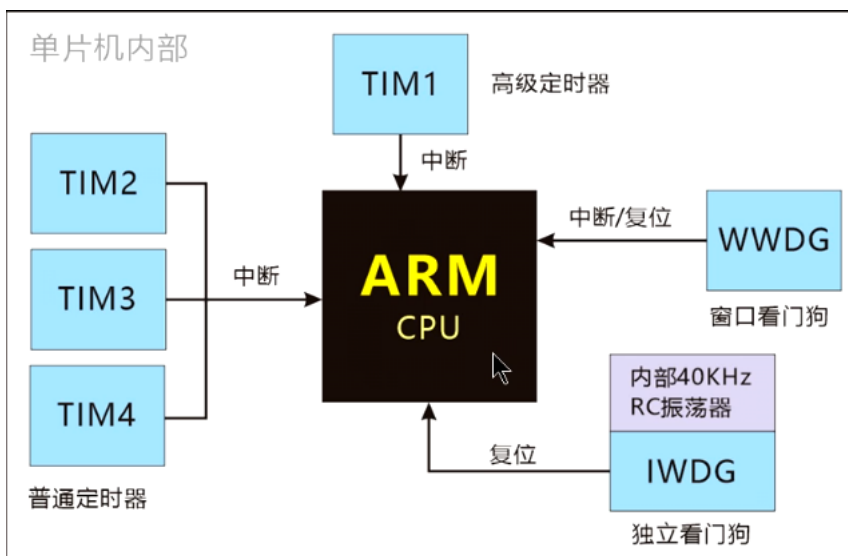
ADC-DR :Analog-to-Digital Converter-Data Register 模拟-数字转换器 数据寄存器

CR:Configuration register 配置寄存器

[参考例程](#)

[ADC](#)

WDG



独立看门狗

独立的看门狗是基于一个12位的递减计数器和一个8位的预分频器，它由一个内部独立的40kHz的RC振荡器提供时钟；因为这个RC振荡器独立于主时钟，所以它可运行于停机和待机模式。它可以被当成看门狗用于在发生问题时复位整个系统，或作为一个自由定时器为应用程序提供超时管理。通过选项字节可以配置成是软件或硬件启动看门狗。在调试模式下，计数器可以被冻结。

窗口看门狗

窗口看门狗内有一个7位的递减计数器，并可以设置成自由运行。它可以被当成看门狗用于在发生问题时复位整个系统。它由主时钟驱动，具有早期预警中断功能；在调试模式下，计数器可以被冻结。

RTC

RTC:Real_Time Clock 实时时钟

32.768kHz 是对应 1 秒

通讯功能

同步通讯

把时钟信号带入特性方程得到数据状态方程

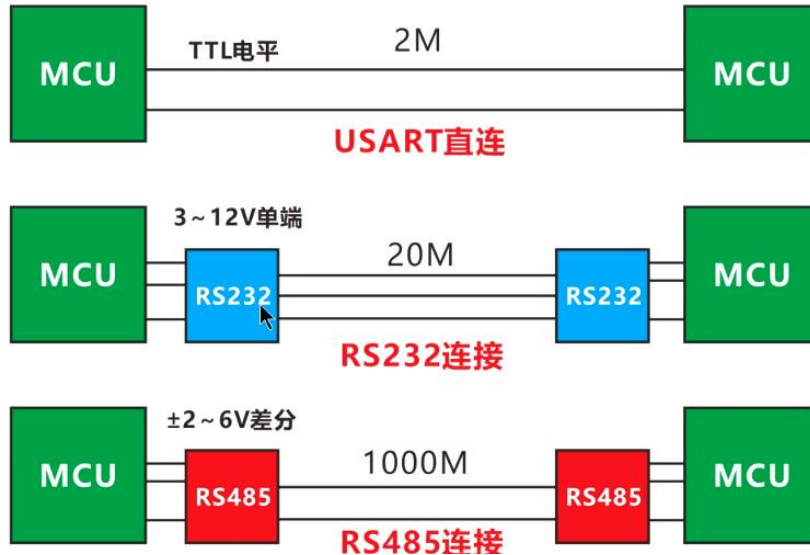
USART

2.3.17 通用同步/异步收发器(USART)

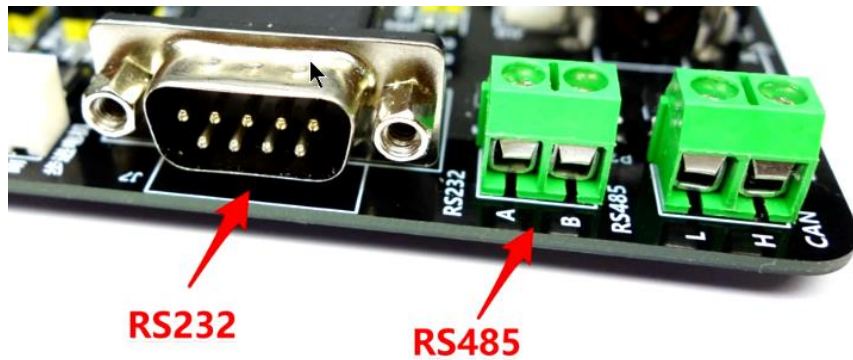
USART1接口通信速率可达4.5兆位/秒，其他接口的通信速率可达2.25兆位/秒。USART接口具有硬件的CTS和RTS信号管理、支持IrDA SIR ENDEC传输编解码、兼容ISO7816的智能卡并提供LIN主/从功能。

所有USART接口都可以使用DMA操作。

- ✓ USART是通用同步/异步收发器（带同步时钟线USART_CK）
- ✓ UART是通用异步收发器（没有同步时钟线）
- ✓ 但最常用的是异步模式，同步模式很少用，所以二者区别不大。
- ✓ USART只是一种协议方式，根据不同电平方式分为RS232和RS485等



RS232&RS485



[串口、COM 口、TTL、RS-232 的区别详解](#)

SPI

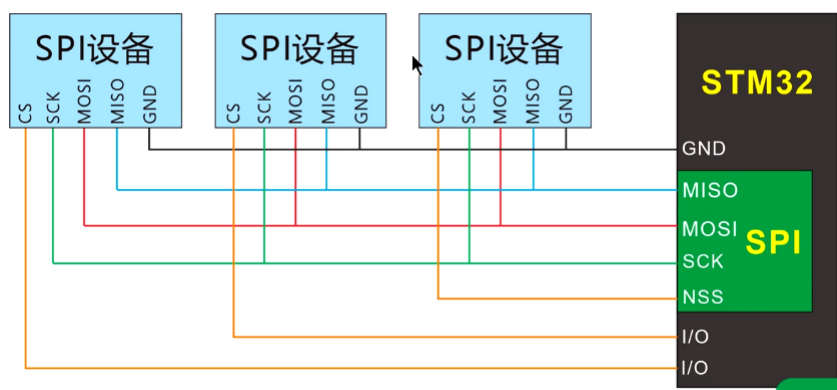
串行外设接口

Serial peripheral interface

SPI用于板级设备间通信

SPI的特点：

- 协议简单稳定
- 速度较快



CAN

Controller Area Network

2.3.19 控制器区域网络(CAN)

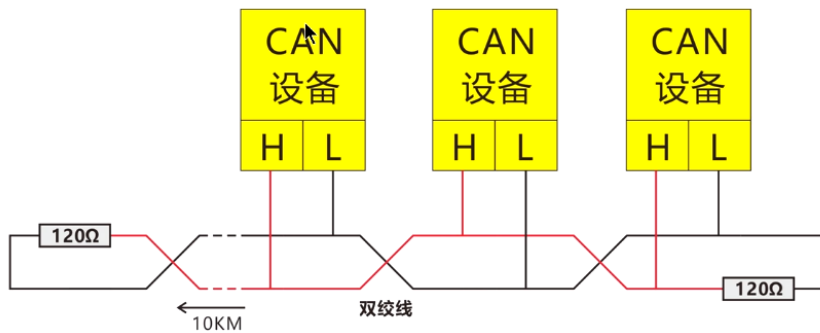
CAN接口兼容规范2.0A和2.0B(主动)，位速率高达1兆位/秒。它可以接收和发送11位标识符的标准帧，也可以接收和发送29位标识符的扩展帧。具有3个发送邮箱和2个接收FIFO，3级14个可调节的滤波器。

- ✓ 有1个CAN总线
- ✓ 位速度最高1M位/S
- ✓ 11位标识符
- ✓ 29位扩展帧
- ✓ 3个发送邮箱
- ✓ 2个FIFO
- ✓ 3级14个滤波器

CAN用于汽车、工业的智能设备通信

CAN的特点是：通信速度快、距离远、稳定、自动查错。

Young



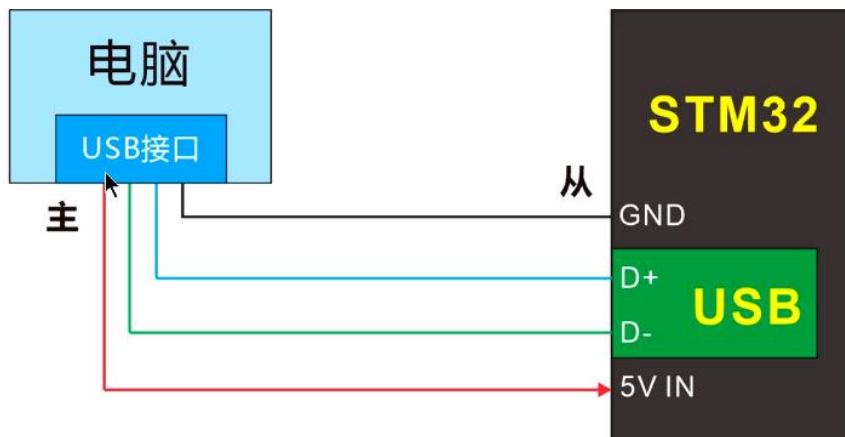
USB

2.3.20 通用串行总线(USB)

STM32F103xx增强型系列产品，内嵌一个兼容全速USB的设备控制器，遵循全速USB设备(12兆位/秒)标准，端点可由软件配置，具有待机/唤醒功能。USB专用的48MHz时钟由内部主PLL直接产生(时钟源必须是一个HSE晶体振荡器)。

- ✓ 1个USB接口
- ✓ 设备控制器
- ✓ 支持全速2.0, 12M位/S
- ✓ 有待机和唤醒功能
- ✓ 由内部PLL倍频器提供时钟
- ✓ 时钟必须由外部高速晶振产生

USB接口用于做PC机的从设备如鼠标、键盘、打印机之类。



CRC

2.3.3 CRC(循环冗余校验)计算单元

CRC(循环冗余校验)计算单元使用一个固定的多项式发生器，从一个32位的数据字产生一个CRC码。在众多的应用中，基于CRC的技术被用于验证数据传输或存储的一致性。在EN/IEC 60335-1标准的范围内，它提供了一种检测内存存储器错误的手段，CRC计算单元可以用于实时地计算软件的签名，并与在链接和生成该软件时产生的签名对比。

- ✓ CRC是用于数据正确性校验的
- ✓ 由一个32位的数据字产生
- ✓ 可应用在FLASH检测
- ✓ 可用于软件签名及对比

